

Time series with MongoDB



Paoletti Alessia · Follow

Published in Data Reply IT | DataTech · 9 min read · Oct 5, 2023



380



Image from <https://www.percona.com/blog/mongodb-5-0-time-series-collections/>

Time series data is everywhere, but storing and querying it can be challenging. MongoDB added native support for time series data making it even easier, faster, and cheaper to work with time series data.

In this article, after a brief introduction on what a time series is, we will introduce **Time Serie Collections** provided by MongoDB from version 5.0.

What is a time series?

A time series represents a series of observations of well-defined data items obtained through repeated measurements over time. Each entry is preceded and followed by another and has a **timestamp** that determines the order of the data.

Nowadays times series are everywhere:



your smartphone daily registers your heart rate variability



retail stores analyze their total sales trending



stock traders use historical data to gain a better understanding of the pattern in various stock prices



weatherman analyses data to predict what temperatures will be during different months and seasons throughout the year

Time series are mostly used to learn what happened, when happened and why happened in order to try to predict what, where and when it will be happen in the future.

Time series data is generally composed of *three components*:

1. *Time* when the data point was recorded
2. *Metadata* (also know as source), which is a label or tag that uniquely identifies a series and rarely changes
3. *Measurements* (also know as metrics or values), which are the data points tracked at increments in time (seconds, minutes, hours, days, months, etc.) Generally these are key-value pairs that change over time.

For example in a weather data scenario, the metric could be the temperature while the metadata could be the sensor identifier and the location.

Time series data is often ingested in massive volumes and requires a **purpose-built-database** that can offer scalability and usability. Scalability is needed because time series data grows very quickly; usability means that you may benefit from built-in functions and features to make data analysis tasks easier.

MongoDB Time Serie Collections

MongoDB is a document database used to build highly available and scalable applications. It uses a JSON-like format to store documents, making easy for developers to store structured or unstructured data.

From version 5.0 MongoDB offers a specific type of collection to efficiently store time series, the so called **Time Series Collections**.

In this type of collections, writes are organized so that data from the same source is stored alongside other data points from a similar point in time.

Time series collections use an underlying columnar storage format and store data in time-order with an automatically created clustered index.

A *columnar storage format* stores all values of a column as a single record. That means all columns for the dataset are stored in a contiguous row; hence, a row in a columnar storage represents all values for that column.

The benefits of columnar storage are:

- reduced complexity for working with time series data
- improved query efficiency
- reduced disk usage
- reduced I/O for read operations

MongoDB treats time series collections as *writable non-materialized views* backed by an internal collection. When you insert data, the internal collection automatically organizes time series data into an optimized storage format. When you query time series collections, you operate on one document per measurement. Queries on time series collections take advantage of the optimized internal storage format and return results faster.

When you create a time series collection, MongoDB automatically creates an internal *clustered index* on the time field. The internal index provides

several performance benefits including improved query efficiency and reduced disk usage. To learn more about the performance benefits of clustered indexes, see [Clustered Collections](#).

Time series collections works as normal collections, you can insert, modify, delete and query your data as you normally do.

Create, populate and query time series collections

To **create** a time series collection you can use `db.createCollection()` command and specify the following time series related parameters:

- *timeField*, the name of the field that contains the date in each time series document.
- *metaField* (optional), the name of the field that contains metadata in each time series document.
- *granularity* (optional), can be “seconds”, “minutes”, “hours”. By default is set to “seconds” for high-frequency ingestion.

```
# Create 'weather' collection
db.createCollection(
  "weather",
  {
    timeseries: {
      timeField: "timestamp",
      metaField: "metadata",
      granularity: "hours"
    }
  }
)
```

```
}  
)
```

Once you have created the collection you can **populate** it. Each measurement inserted should be a single measurement, either as individual documents or batches of documents with one measure per document. MongoDB optimizes the data, as it stores data ordered by time (as opposed to normal collections where data are stored in “natural order”, that usually coincide with the inserting order).

```
# Insert many documents at once in the collection  
db.weather.insertMany( [  
  {  
    "metadata": { "sensorId": 5578, "type": "temperature" },  
    "timestamp": ISODate("2021-05-18T00:00:00.000Z"),  
    "temp": 12  
  },  
  {  
    "metadata": { "sensorId": 5578, "type": "temperature" },  
    "timestamp": ISODate("2021-05-18T04:00:00.000Z"),  
    "temp": 11  
  },  
  {  
    "metadata": { "sensorId": 5578, "type": "temperature" },  
    "timestamp": ISODate("2021-05-18T08:00:00.000Z"),  
    "temp": 11  
  },  
  {  
    "metadata": { "sensorId": 5578, "type": "temperature" },  
    "timestamp": ISODate("2021-05-18T12:00:00.000Z"),  
    "temp": 12  
  }  
] )
```

On the [documentation](#) you can find some best practice useful when you need to create a time series collection.

Once data are in the collection you can easily **query** them in many different ways.

You can use `db.collection.findOne()` or `db.collection.findMany()` to retrieve one or more documents according to the given input filters. You can retrieve a specific measurement or multiple observations in a specific range of time.

```
# Find a specific document using its timestamp
db.weather.findOne({
  "timestamp": ISODate("2021-05-18T00:00:00.000Z")
})
```

As in any other MongoDB collections you can perform **aggregation pipeline**. Aggregation pipelines process multiple documents and return computed results. Thanks to aggregation you can group multiple documents together; perform operations on the grouped data to return a single result; analyze data changes over time.

```
# Aggregate pipeline to compute the average daily temperature
db.weather.aggregate( [
  {
    # 1. Split timestamp to date component (year, month, day)
```

```
$project: {
  date: {
    $dateToParts: { date: "$timestamp" }
  },
  temp: 1
}
},
{
  # 2. Group data according to the date
  $group: {
    _id: {
      date: {
        year: "$date.year",
        month: "$date.month",
        day: "$date.day"
      }
    },
    # 3. Compute average temperature using "temp" field
    avgTmp: { $avg: "$temp" }
  }
}
] )
```

To improve query performance **secondary indexes** on the *metaField* and/or the *timeField* can be added.

Automatic delete or archiving documents

After a certain time period time series data might not be so relevant and you might want to delete or archive old data to keep the collections as small as possible, improving performance and reducing costs.

If you would like to remove data after a certain time has passed you can use the *expireAfterSeconds* option that enable the automatic deletion of documents by specifying the number of seconds after which documents expire. This parameter can be set or modified at any point during or after collection creation, so if the requirement change, you can easily modify

or update the Time To Live (TTL) for your date.

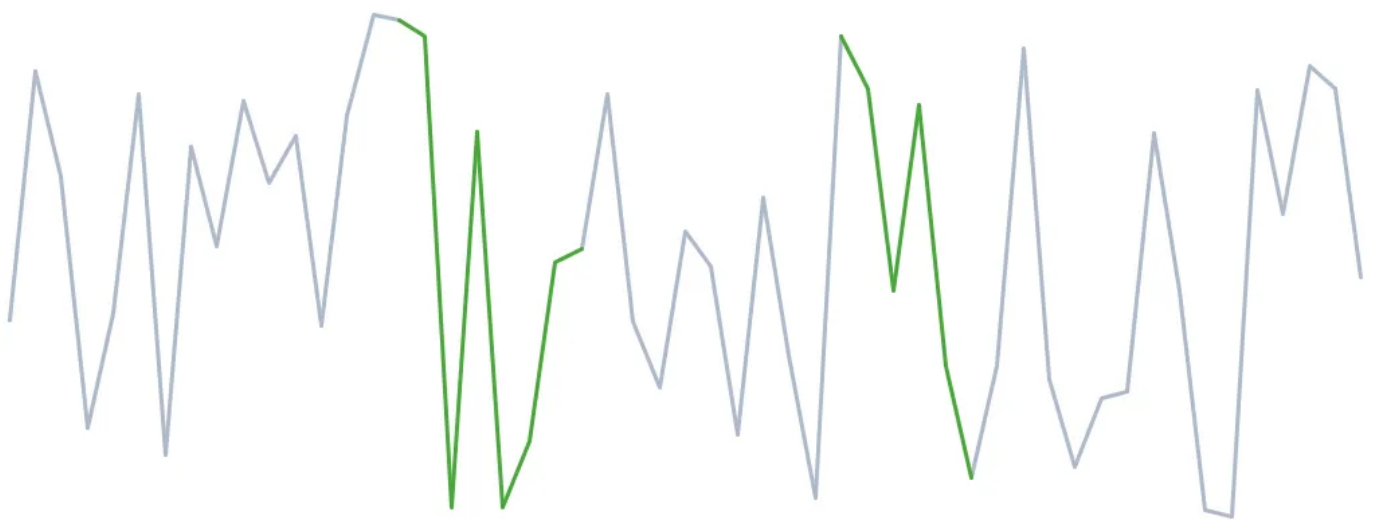
```
# Create 'weather' collection with TTL option
db.createCollection(
  "weather",
  {
    timeseries: {
      timeField: "timestamp",
      metaField: "metadata",
      granularity: "hours"
    },
    expireAfterSeconds: 86400 # TTL option
  }
)

# or set/modify TTL option after collection's creation
db.runCommand({
  collMod: "weather",
  expireAfterSeconds: 86400 # TTL option
})
```

Another option for handling old data is to tier it into operational and online archive storage. Using *Atlas Online Archive* you can automatically archive data from your Atlas cluster to a MongoDB managed Data Lake. For details information see the official MongoDB documentation on [how configuring online archives](#).

Deal with missing data

Time series data frequently exhibit gaps for a number of reasons, including measurement failures, formatting problems, human errors or a lack of information to record. On the other hand, time series data must be continuous to perform analytics and guarantee accurate outcome.



MongoDB time series interpolation

From version 5.3 MongoDB introduces **gap filling**, to make easier to handle missing data and to allow to bring valuable insight to light. Gap filling is possible thanks to `$densify` and `$fill` commands.

The `$densify` command creates new documents to eliminate the gaps in the time or numeric domain at the required granularity level.

The `$fill` command provides the actual values or data for the new entries. Missing data points can be populated with a fixed values or with a value based on the surrounding documents using linear interpolation or using the value of last or next observation.

Given a collection with the following two documents, representing the temperature of a given sensor at 8AM and 10AM of 18th of May:

```
[
  {
    "metadata": { "sensorId": 5578, "type": "temperature" },
    "timestamp": ISODate("2021-05-18T08:00:00.000Z"),
    "temp": 10
  },
  {
    "metadata": { "sensorId": 5578, "type": "temperature" },
    "timestamp": ISODate("2021-05-18T10:00:00.000Z"),
    "temp": 15
  }
]
```

This function produces an hourly view of the temperature. When temperature data is missing it should be interpolated linearly:

```
[
  { $densify: {
    field: "timestamp",
    partitionByFields: ["metadata.sensorId"],
    range: {
      step: 1,
      unit: "hour",
      bounds: "partition"
    }
  } },
  { $fill: {
    partitionByFields: ["metadata.sensorId"],
    sortBy: {"timestamp": 1},
    output: {
      temp: {method: "linear"}
    }
  } }
]
```

In this way the following document is created at 9AM with a value of 12.5 for the field temperature:

```
{
  "metadata": { "sensorId": 5578, "type": "temperature" },
  "timestamp": ISODate("2021-05-18T09:00:00.000Z"),
  "temp": 12.5
}
```

Medium

Search



Write

Sign
upSign
in

TIME SERIES COLLECTIONS LIMITATIONS

Time series collection have some limitations:

- some feature like Atlas Search, GraphQL API, database triggers are not supported;
- you cannot use `$out` or `$merge` aggregation pipeline stages to add data from another collection to a time series collection;
- you can only set the collection type when you create a collection: an existing collection cannot become a time series collection and, in the same way, a time series collection cannot be converted into a different collection type;
- you cannot write to time series collections in transactions;
- time series collections are *writable non-materialized views*. Limitations for views apply to time series collections.

At the following [link](#) all time series limitations are reported in details.

Code example with PyMongo

MongoDB offers **drivers** for all major programming languages, so you don't have to waste time configuring a database. Check the documentation to discover all [supported drivers](#).

In the following example we will use Python as programming language and [PyMongo](#) as MongoDB driver.

MongoDB Community Edition can be installed following this [guide](#). Make sure to install version 5.3 or later in order to use time series collection and all related functions.

In our example we will consider daily weather measurement in the city of Bologna for the month of July. Each measurement is made up of day, temperature, millimeters of rain, percentage of humidity. Data is uncompleted: temperature, rain and/or humidity data was not registered for some days.

Here an example of the data for the first five days and the documents that will be inserted in the MongoDB time series collection:

timestamp	temp	rain	humidity
7/1/2023	40		58
7/2/2023	38		80
7/3/2023		200	
7/4/2023	31		87
7/5/2023	35		85

```
{'timestamp': Timestamp('2023-07-01 00:00:00'), 'metadata': {'sensor': '1234', 'city': 'Bologna'}, 'temp': 40.0, 'humidity': 58.0}
{'timestamp': Timestamp('2023-07-02 00:00:00'), 'metadata': {'sensor': '1234', 'city': 'Bologna'}, 'temp': 38.0, 'humidity': 80.0}
{'timestamp': Timestamp('2023-07-03 00:00:00'), 'metadata': {'sensor': '1234', 'city': 'Bologna'}, 'rain': 200.0}
{'timestamp': Timestamp('2023-07-04 00:00:00'), 'metadata': {'sensor': '1234', 'city': 'Bologna'}, 'temp': 31.0, 'humidity': 87.0}
{'timestamp': Timestamp('2023-07-05 00:00:00'), 'metadata': {'sensor': '1234', 'city': 'Bologna'}, 'temp': 35.0, 'humidity': 85.0}
```

```
import pymongo
import pandas as pd

# Connect to the database
client = pymongo.MongoClient('mongodb://localhost:27017/')

# Get database 'italyWeather' (create if does not exist)
db = client.italyWeather

# Create time series collection
collection_name = 'bolognaWeather'
db.create_collection(
    collection_name,
    timeseries= {
        "timeField": "timestamp",
        "metaField": "metadata",
        "granularity": "hours" # seconds, minutes or hours
    }
)

# Select the collection to use
collection = db['bolognaWeather']
```

If the collection “*bolognaWeather*” already exists, the command

db.create_collection will throw an error.

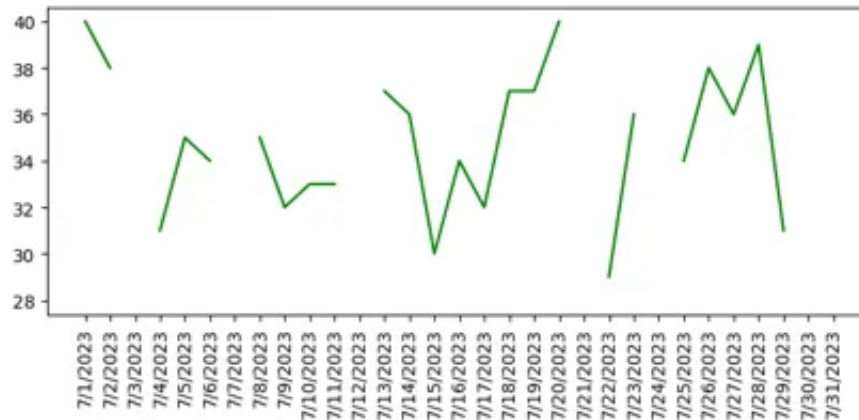
```
# Insert the list of observation into the collection
collection.insert_many([
{'timestamp': Timestamp('2023-07-01 00:00:00'), 'metadata': {'sensor': '1234'}
{'timestamp': Timestamp('2023-07-02 00:00:00'), 'metadata': {'sensor': '1234'}
{'timestamp': Timestamp('2023-07-03 00:00:00'), 'metadata': {'sensor': '1234'}
  # ... all the remaining days....
])
```

Due to missing data we can perform `$fill` command to fill them. This command does not update the documents in the database, but return a list of new documents filled with the missing data according to the chosen method.

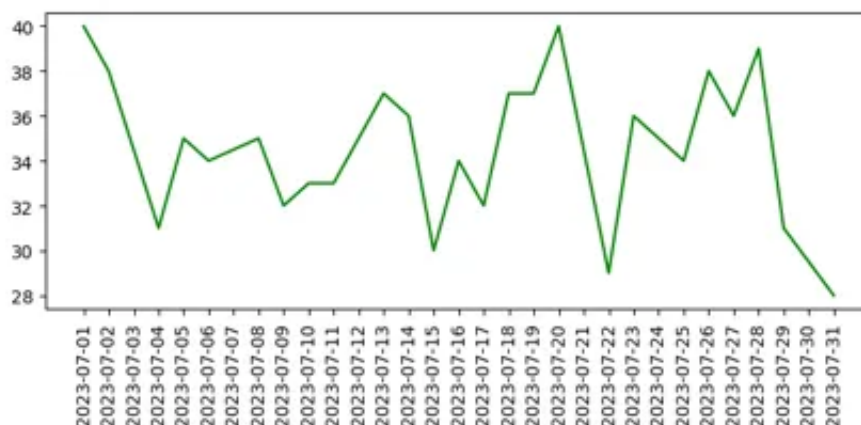
```
# Perform the fill stage choosing the appropriate method for each field:
# temperature is filled using linear interpolation
# rain is filled with constant = 0 (ideally, no data means no rain)
# humidity is filled with the value of the backward observation

collection.aggregate( [
  {
    "$fill":
    {
      # data is sorted according to the timestamp
      "sortBy": { "timestamp": 1 },
      "output":
      {
        "temp": { "method": "linear" },
        "rain": { "value": 0 },
        "humidity": { "method": "locf" }
      }
    }
  }
] )
```

Original temperature data
(with missing values)



Temperature data after linear interpolation



Thanks to fill stage we have “complete” data on from which we can extract meaningful highlights.

Conclusions

Nowadays deal with time series data is very common. Time series collection offered by MongoDB allows you to deal with this type of data in an easy and efficient way, letting you concentrate in gaining all the value of the data.

References

- [1] [Introducing Gap Filling for Time Series Data in MongoDB 5.3](#)
- [2] [All Data Is Time-Series Data \(With Examples\)](#)
- [3] [MongoDB docs — Time Series](#)
- [4] [Time Series Collection Limitations](#)
- [5] [Understanding Columnar vs Row Storage Formats for Data Analytics and Reporting](#)
- [6] [Best Practices for Time Series Collections](#)

Mongodb

Time Series Collections



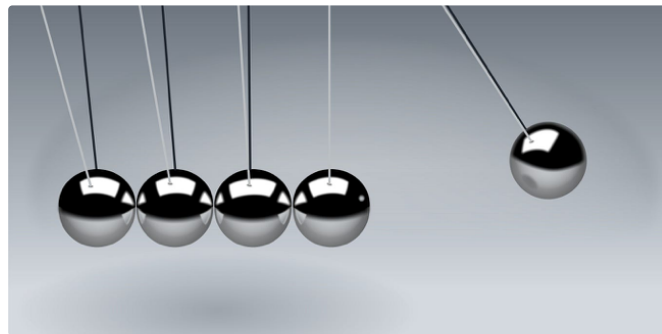
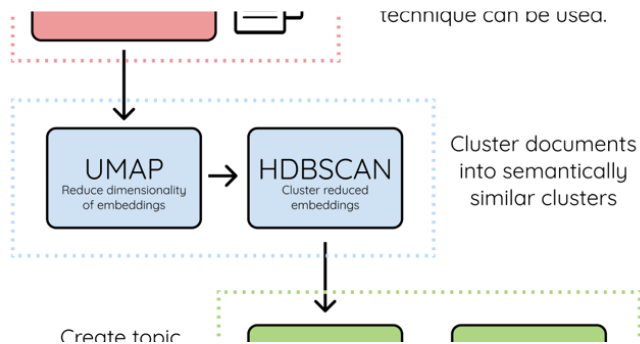
Written by Paoletti Alessia

5 Followers · Writer for Data Reply IT | DataTech

Follow



More from Paoletti Alessia and Data Reply IT | DataTech



Elisa Distanti in Data Reply IT | DataTech

BERTopic: topic modeling as you have never seen it before

NLP (Natural Language Processing) is one of the most complex fields of Artificial...

Oct 20, 2022



282



1



Andrea Mosca in Data Reply IT | DataTech

Uplift Modeling: predict the causal effect of marketing...

A complex task in marketing analytics is the skill to predict customers' reactions to...

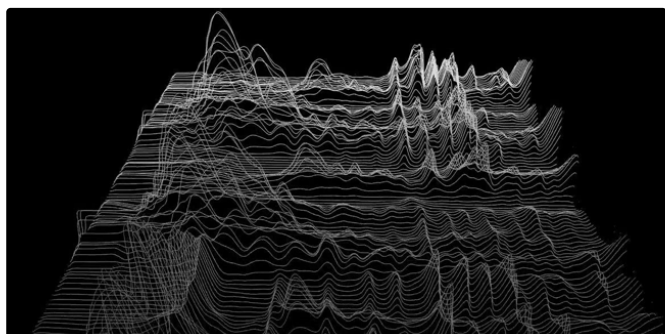
Nov 30, 2023



142



2



Cristian Urbinati in Data Reply IT | DataTech

Spatio-Temporal Forecasting using Temporal Graph Neural...

Application of Spatio-Temporal GNN for traffic forecasting on a custom dataset...

Jun 5, 2023



102



Fernando De Nitto in Data Reply IT | DataTech

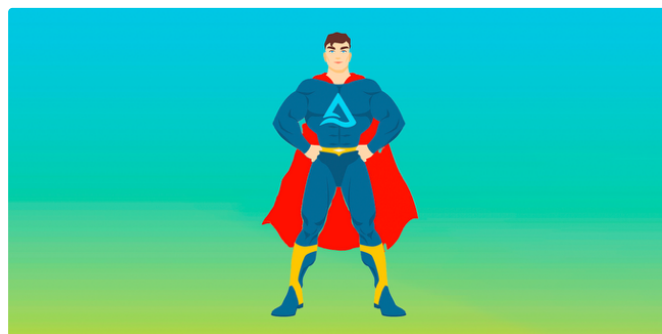
Mastering Delta Lake Optimizations: OPTIMIZE, Z-...

Find out how to use the optimizations of your Delta Lake to increase the...

Sep 28, 2023



417

[See all from Paoletti Alessia](#)[See all from Data Reply IT | DataTech](#)

Recommended from Medium



Ramachandran Krish

UUID vs ULID vs UUID7: Why, Where, and When?

ULID (Universally Unique Lexicographically Sortable Identifier) History and Algorithm:...

Jul 1



Jun 1



25K



511



Alexander Nguyen in Level Up Coding

The resume that got a software engineer a \$300,000 job at...

1-page. Well-formatted.

Lists



Natural Language Processing

1815 stories · 1427 saves



 Amit Verma

MongoDB 101: A Practical Guide to NoSQL Databases

From SQL to NoSQL: Transitioning to MongoDB for Modern Applications.

Aug 12  27



 Abdur Rahman in Stackademic

Python is No More The King of Data Science

5 Reasons Why Python is Losing Its Crown

★ Oct 23  6.6K  29



Cassandra and MongoDB



 Daniel Po

Cassandra and MongoDB

What's the difference between Cassandra and MongoDB?

★ Jun 24  1



 Hussein Nasser

The Art of database systems

The art of truly understanding databases boils down to the following principles.

★ Jul 25  302



See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Terms](#) [Text to speech](#) [Teams](#)